# SDG BLOCKCHAIN ACCELERATOR

## Technical Architecture Document

## TradeChainx & UNDP Ethiopia

- **Project Name:**
  Blockchain-Based Carbon Credits MRV Platform

- **Challenge & UNDP Office**
  SDG Blockchain Accelerator

- **Document Version** v1.0

## 2. Overview

This platform implements a **hybrid on-chain/off-chain MRV (Measurement, Reporting, and Verification) system** for carbon credit projects. The system records the full lifecycle of a project on the Cardano blockchain while handling data-intensive MRV processes off-chain.

The architecture ensures:

- Deterministic lifecycle enforcement via **eUTxO state transitions**
- Cryptographic linkage between MRV data and on-chain state
- Controlled issuance of carbon credits based on verified data
- Irreversible retirement to eliminate double counting

The design deliberately separates **data availability** from **state validity**:

- **Off-chain systems** manage data ingestion, storage, and processing
- **On-chain logic** enforces correctness, permissions, and token lifecycle

## 3. System Architecture

### 3.1 High-Level Architecture

```
                    ┌──────────────────────────────┐
                    │        React Frontend        │
                    │  (Developer / Verifier UI)   │
                    └──────────────┬───────────────┘
                                   │
                        Wallet Connection
                       (Nami / Lace CIP-30)
                                   │
                    ┌──────────────▼───────────────┐
                    │     Lucid (Tx Builder)       │
                    │  - Build Tx                  │
                    │  - Select UTxOs              │
                    │  - Submit Tx                 │
                    └──────────────┬───────────────┘
                                   │
              ┌────────────────────▼────────────────────┐
              │        Cardano Blockchain               │
              │-----------------------------------------│
              │  Aiken Validators:                      │
              │  - Project Registry                     │
              │  - Lifecycle Validator                  │
              │  - Marketplace Validator                │
              │                                         │
              │  Minting Policy:                        │
              │  - Mint / Burn Carbon Tokens            │
              └────────────────────┬────────────────────┘
                                   │
        ┌──────────────────────────┼──────────────────────────┐
        │                          │                          │
┌───────▼────────┐        ┌────────▼────────┐
┌───────▼────────┐
│   Node Backend │        │     Supabase DB     │        │      IPFS        │
│----------------│        │---------------------│        │------------------│
│ - MRV Workflow │        │ - MRV structured    │        │ - Reports        │
│ - Role Mgmt    │        │ - Users             │        │ - Metadata       │
│ - Validation   │        │ - Audit logs        │        │ - Proof storage  │
└────────────────┘        └─────────────────────┘
└────────────────┘
```

### 3.2 Component Responsibilities

| Component | Responsibility |
|---|---|
| React Frontend | User interaction, wallet integration |
| Wallet (CIP-30) | Transaction signing |
| Lucid | Transaction construction |
| Aiken Validators | Enforce lifecycle rules |
| Minting Policy | Control issuance and burning |
| Backend (Node.js) | MRV workflow + orchestration |
| Supabase | Structured data + audit |
| IPFS | Immutable document storage |

### 3.3 Interaction Model

| Actor | Interaction |
|---|---|
| Developer | Registers project, submits MRV |
| Verifier | Approves/rejects MRV |
| Buyer | Purchases and retires credits |

## 4. Blockchain Design

### 4.1 Contract Architecture

| Contract | Type | Function |
|---|---|---|
| Project Registry | Validator | Creates project state |
| Lifecycle Validator | Validator | Controls transitions |
| Minting Policy | Policy | Mint/burn tokens |
| Marketplace | Validator | Trade execution |

### 4.2 State Machine

```
┌──────────────────┐
│   REGISTERED     │
└──────────────────┘
        │ SubmitReport
        ▼
┌──────────────────┐
│    REPORTING     │
└──────────────────┘
        │ VerifyProject
        ▼
┌──────────────────┐
│    VERIFIED      │
└──────────────────┘
        │ MintCredits
        ▼
┌──────────────────┐
│     MINTED       │
└──────────────────┘
        │ RetireCredits
        ▼
┌──────────────────┐
│     RETIRED      │
└──────────────────┘
```

### 4.3 Datum Structure

```
ProjectDatum {
 project_id: ByteArray
 owner: Address
 state: Enum
 verified_amount: Integer
 metadata_hash: ByteArray
 verifier: Address
 timestamp: Integer
}
```

### 4.4 Redeemer Structure

```
RegisterProject
SubmitReport
VerifyProject
MintCredits
ListCredits
BuyCredits
TransferCredits
RetireCredits
```

### 4.5 UTxO Model

| Concept | Implementation |
|---|---|
| Project | Single state UTxO |
| State change | Consume + recreate |
| Ownership | Wallet UTxOs |
| Tokens | Native assets |

### 4.6 Token Model

| Property | Value |
|---|---|
| Unit | 1 token = 1 ton $CO_2e$ |
| Minting | Verified only |
| Burning | Required for retirement |
| Supply | Dynamic |

### 4.7 Validation Logic Example

```
validate(tx):
 assert datum.state == VERIFIED
 assert tx.signed_by(verifier)
 assert tx.minted == datum.verified_amount
```

# 5. Data Flow & Transaction Lifecycle

## 5.1 End-to-End Flow

```
[1] Register Project
Frontend → Lucid → Blockchain
→ Create Project UTxO

[2] Submit MRV Data
Frontend → Backend → Supabase
→ Pin to IPFS → store hash

[3] Verify Project
Verifier → Submit Tx
→ Update state to VERIFIED

[4] Mint Credits
Minting Policy → Issue tokens
→ Assign to developer

[5] Marketplace Trade
Buyer → Purchase Tx
→ Transfer ownership

[6] Retirement
Buyer → Burn Tx
→ Record offset event
```

## 5.2 Data Flow Model

```
MRV Data → Supabase (structured)
        → IPFS (immutable)

IPFS Hash → Stored in Datum

Blockchain → Source of truth
```

# 6. Off-chain Architecture

## 6.1 Backend Services

| Service | Function |
|---|---|
| MRV Engine | Validate reports |
| Role Manager | Verifier permissions |
| Tx Listener | Sync blockchain events |
| API Layer | Serve frontend |

## 6.2 Storage Design

| Data | Location |
|---|---|
| MRV datasets | Supabase |
| Reports | IPFS |
| State references | Blockchain |

# 7. Security Architecture

## 7.1 Access Control

| Action | Requirement |
|---|---|
| Register | Owner signature |
| Verify | Authorized verifier |
| Mint | Verified state |
| Transfer | Token ownership |

### 7.2 Threat Mitigation

| Threat | Mitigation |
|---|---|
| Double minting | UTxO consumption |
| Replay attack | State transitions |
| Fake MRV | Verifier validation |
| Data tampering | IPFS hashing |

### 7.3 Trust Model

- Blockchain = **enforcement layer**
- Backend = **processing layer**
- Verifiers = **controlled trust layer**

## 8. Performance & Scalability

### 8.1 Constraints

| Constraint | Impact |
|---|---|
| Execution units | Script limits |
| UTxO size | Datum size |
| Throughput | L1 limits |

### 8.2 Optimization

- Minimal datum encoding
- Batch minting
- Off-chain aggregation
- Hydra (future scaling)

## 9. Testnet Results

### 9.1 Functional Validation

| Feature | Result |
|---|---|
| Registration | Success |
| Verification | Success |
| Minting | Success |
| Transfer | Success |
| Retirement | Success |

### 9.2 Observations

- No double minting
- No unauthorized actions
- Stable execution costs

## 10. Tools & Environment

| Layer | Tool |
|---|---|
| Blockchain | Cardano Preprod |
| Contracts | Aiken |
| Tx Layer | Lucid Evolution |
| Frontend | React |
| Backend | Node.js |
| DB | Supabase |
| Storage | IPFS |
| Wallets | Nami, Lace |

## 11. Future Enhancements

### 11.1 Governance

- Verifier onboarding DAO

- Reputation scoring

### 11.2 MRV Automation

- IoT sensors
- Satellite feeds
- Oracle integration

### 11.3 Compliance

- Verra alignment
- Gold Standard compatibility

### 11.4 Advanced Features

- Carbon credit pools (ETF-style)
- Cross-chain bridging
- Secondary liquidity markets

## 12. Key Differentiators

- State-enforced lifecycle via UTxO
- On-chain/off-chain verifiable linkage
- Deterministic minting tied to MRV
- Provable retirement (burn mechanism)